

Week 6, Lecture 2 WOD: Answer Key

Find the Bugs

NRES 746, Instructor Answer Key (not for student distribution)

Each item repeats the handout's question, then gives the answer, a short rationale, and mistakes to watch for. Every buggy function is run below, so the symptoms described are real R output. The corrected functions match the Lab 4, Act 2 key, and they are the ones built live in today's demo.

Part 1: Find and fix

1a.

`detect_prob()`: circle the bug, write the corrected line, and say what would go wrong if you ran the buggy version.

Answer. `1 - pobs^k` should be `1 - (1 - pobs)^k`.

Why. An animal is missed on one person-day with probability $1 - p_{obs}$, and missed on all k days with probability $(1 - p_{obs})^k$. The buggy line uses the chance of being *detected* every day instead. The result is a detection probability near 1, so the simulated counts would be close to the true abundance:

```
detect_prob_bug <- function(k, pobs = 0.02) 1 - pobs^k
c(buggy = detect_prob_bug(3), fixed = detect_prob(3))
```

```
##      buggy      fixed
## 0.999992 0.058808
```

Watch for. Writing `(1 - pobs)^k` without the leading `1 -`. That is the probability of being missed, not detected.

1b.

`simulate_counts()`: circle the bug, write the corrected line, and say what would go wrong if you ran the buggy version.

Answer. `rbinom(1, ...)` should be `rbinom(length(t_survey), ...)`.

Why. The first argument of `rbinom()` is how many draws to make. With 1, the function returns a single count (using only the first year's abundance) instead of one count per survey year. `lm()` then fails because `counts` and `t_survey` have different lengths:

```
simulate_counts_bug <- function(t_survey, k, NO = 1000, lambda = 0.75^(1/25)) {
  N_t <- NO * lambda^t_survey
  rbinom(1, size = round(N_t), prob = detect_prob(k))
}
set.seed(1)
simulate_counts_bug(t1, k = 3)
```

```
## [1] 60
```

```
tryCatch(detect_decline(t1, simulate_counts_bug(t1, k = 3)),
  error = function(e) conditionMessage(e))
```

```
## [1] "variable lengths differ (found for 't_survey')"
```

Watch for. Students who say “size should be N0.” That would remove the decline entirely. `size` must follow `N_t`.

1c.

`detect_decline()`: circle the bug, write the corrected line, and say what would go wrong if you ran the buggy version.

Answer. `||` (or) should be `&&` (and): `s[1] < 0 && s[4] <= alpha`.

Why. With “or,” any negative slope counts as a detected decline, significant or not, and so does any significant *increase*. Under a stable population ($\lambda = 1$), about half of all slopes are negative by chance, so the buggy rule “detects” a decline roughly half the time instead of about 2.5%:

```
detect_decline_bug <- function(t_survey, counts, alpha = 0.05) {  
  s <- summary(lm(log(counts) ~ t_survey))$coefficients[2, ]  
  s[1] < 0 || s[4] <= alpha  
}  
set.seed(2)  
null_sims <- replicate(1000, simulate_counts(t1, k = 3, lambda = 1), simplify = FALSE)  
c(buggy = mean(sapply(null_sims, function(y) detect_decline_bug(t1, y))),  
  fixed = mean(sapply(null_sims, function(y) detect_decline(t1, y))))
```

```
## buggy fixed  
## 0.523 0.028
```

Watch for. Changing `<=` to `<`. That’s a harmless difference, not the bug.

1d.

`estimate_power()`: circle the bug, write the corrected line, and say what would go wrong if you ran the buggy version.

Answer. `detected <- ...` should be `detected[i] <- ...`

Why. Without the index, each pass through the loop overwrites the whole vector with a single `TRUE` or `FALSE`. After the loop, `detected` holds only the last replicate, so the “power” is always exactly 0 or 1:

```
estimate_power_bug <- function(k, s, R = 1000) {  
  t_survey <- seq(1, 25, by = s)  
  detected <- logical(R)  
  for (i in seq_len(R)) {  
    counts <- simulate_counts(t_survey, k)  
    detected <- detect_decline(t_survey, counts)  
  }  
  mean(detected)  
}  
set.seed(3)  
replicate(8, estimate_power_bug(3, 1, R = 50))
```

```
## [1] 1 1 0 1 1 1 0 1
```

Watch for. Students who fix it by moving `mean()` inside the loop. That still keeps only one replicate’s result.

Part 2: Catch it next time

Pick one of the bugs. Describe a quick test, one line of code and the answer you’d expect, that would have caught it before you trusted the function.

Sample answers (any one is fine).

- **1a:** `detect_prob(1)` should return 0.02 (one day of effort should equal `pobs`).
- **1b:** `length(simulate_counts(1:25, 3))` should return 25.
- **1c:** run `detect_decline()` many times on data simulated with `lambda = 1`. It should return `TRUE` only about 2-3% of the time.
- **1d:** run `estimate_power(3, 1)` twice. Each should be a value between 0 and 1 (about 0.8 here), not exactly 0 or 1.

Why. Good tests use cases where you already know the answer: an edge case (`k = 1`), a shape check (`length()`), or a scenario with known truth (no decline). These are the same checks built into the Lab 4, Act 2 handout, and the same ones to demand from AI-written code in today's demo.

Watch for. "Run it and see if the answer looks reasonable." Bug 1a gives counts that look like plausible numbers unless you know what to expect. A test needs a specific expected value.